Segment This Thing, Bit by Bit

Maciej Lewandowski[1,*] Shinjeong Kim[2,*] Nicholas Fry[2,*] Piotr Dudek[1]
Paul H. J. Kelly[2] Andrew J. Davison[2]
[1]University of Manchester [2]Imperial College London
{maciej.lewandowski, p.dudek}@manchester.ac.uk
{s.kim, nf20, p.kelly, a.davison}@imperial.ac.uk

Abstract

Inspired by biological vision, where foveated retina and early neural circuits compress visual input before it leaves the eye, we implement a point-directed segmentation pipeline based on the "Segment This Thing" architecture on a custom bit-serial, near-memory SIMD processor array. The hardware is prototyped on an Xilinx KV260 FPGA, and consists of 2,304 processing elements with bit-flexible arithmetic capabilities. We co-design the model and hardware using a Python instruction-set simulator and verify them against a cycle-accurate RTL co-simulation, which enables rapid iteration without synthesis. Foveated tokenisation compresses the input image into just 40 tokens, achieving an 84% token reduction while preserving resolution around the query point. To our knowledge, this is the first implementation of a foveated transformer encoder on a Pixel Processor Array-like design.

1. Introduction

Standard vision pipelines transfer raw pixel data from sensor to off-chip memory before any processing occurs. For robotics and augmented reality systems operating under strict power and latency budgets, this is wasteful: low-level vision algorithms are inherently compressive, and most transferred data is discarded in early processing stages.

Biology offers a better model: vertebrate retinas provide higher resolution in the central region (fovea), reducing information at the periphery, and concentrating on objects of interest through attention-driven foveation. Neural circuits are distributed across the retina, near the photoreceptors, providing extraction and encoding of visual information using local computations, before transmitting data to the rest of the brain. On-sensor computing brings these principles to silicon. Architectures such as SCAMP [5] demonstrate that filtering, feature extraction [15], tracking [4], visual odometry [2], and even recurrent networks [13] can run close to the sensor, transmitting only compact task-relevant representations to the host. This work extends that paradigm with attention-driven foveated processing, similar to the human retina. In this paper we present, to our knowledge, the first foveated transformer encoder running on a near-pixel processor array. Building on Segment This Thing [12] (STT), we introduce foveated tokenisation to encode image regions at variable resolution around a query point, and implement the full encoder (tokenisation, linear embedding, positional encoding, and attention) on a novel bit-serial SIMD processor, prototyped in simulation and on FPGA.

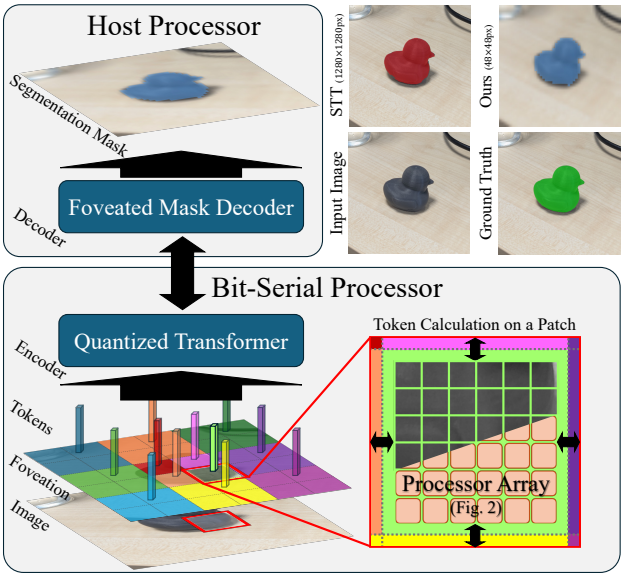


Figure 1. Overview of the proposed processor and our primary workload of interest, Segment This Thing [12] quantized and retrained to fit in the proposed processor.

2. Hardware Architecture

As a prototype for a next-generation on-sensor pixel-processor array, inspired by the SCAMP-5 system [5], we designed a custom digital processor array and implemented it on a Xilinx KV260 FPGA platform. The system diagram

[*]Equal contribution.

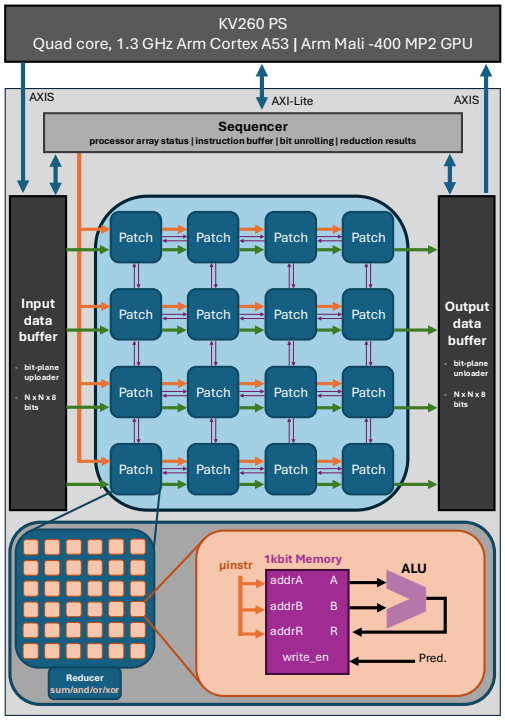


Figure 2. System architecture. The KV260 SoC contains a quad-core ARM A53 at 1.3 GHz that communicates with the programmable logic via an AXI interface. The design comprises three memory buffers (a memory-mapped instruction store and two I/O buffers) and the processor grid. The grid has two levels of hierarchy: patches (light blue) that share memory and word-parallel circuits, each containing 6×6 bit-serial processing elements (light orange).

is shown in Fig. 2. Targeting digital FPGA fabric rather than analogue mixed-signal silicon enables exact reproducibility, rapid architectural iteration, and tight integration with the ARM processing system on the same die.

In its current configuration on the KV260 (utilising a low-cost Zynq Ultrascale+ SoC FPGA), the system consists of a 48×48 grid of processing elements (PEs) organised into an 8×8 array of patches, each containing 6×6 PEs. Every PE has a bit-serial ALU, a 1-bit predicate register, and 1 kbit of local memory. The design is highly parameterised and can be scaled to larger FPGAs by adjusting a small number of parameters, like the number of patches, number of PEs per patch or amount of memory per PE. The PEs operate as a SIMD array: all PEs execute the same instructions, while each PE's predicate register independently controls whether that result is written back to memory. Predicate masks can be set by instructions that generate regular spatial patterns (e.g. selecting specific rows, columns, or a rectangular subregion) or loaded as arbitrary bit-planes from the host processor.

The PEs operate bit-serially: rather than processing fixed-width words in one cycle, they consume operands one bit at a time over multiple cycles, with the ALU maintaining carry state across bit positions. Arithmetic precision is therefore software-defined: the programmer specifies the operand width per instruction, and different stages of the pipeline can operate at different precisions with no hardware cost beyond additional clock cycles. For instance, patch embedding convolutions can use 2-bit ternary weights, reducing each filter to a sequence of additions and subtractions, while attention computation can operate at 8–16 bits and use the DSP-based multiplier shared across each patch. Arbitrary widths such as 3 or 15 bits are equally straightforward. All of these precisions coexist within the same program on the same fabric.

Table 1. Representative instructions. All operations execute across the full PE array in parallel; writes are gated per-PE by the predicate register. The operand width `#b` is specified per instruction, making arithmetic precision software-defined.

Class	Example	Effect
Arithmetic	`ADD Rd[a],Rs[a],#b`	`Rd +=` `Rs`, `#b`-bit
	`SUBI Rd[a],Imm,#b`	`Rd −=` `Imm`, `#b`-bit
Logic	`AND Rd[a],Rs[a],#b`	bitwise and/or/xor/not
Move	`MOVI Rd[a],Imm,#b`	broadcast constant to all PEs
Shift (NEWS)	`MOVN Rd[a],Rs[a],#b`	shift data one PE north
Predicate	`MOVP Rs[a]`	load mask from memory
	`ANDP Rs[a]`	combine mask with current pred.
I/O	`LDG / STG`	transfer to/from host buffer
Reduce	`RED.SUM Rs[a],#b`	sum all predicated PEs
Multiply	`MUL16 Rd[a],Rs[a]`	patch-level DSP multiply

[a] `Rd` and `Rs` denote destination and source addresses : this can be any value from $[0, 1023]$. The operand width `#b` specified how many bits are we processing.
[*] uses shared DSP slice per patch at word-parallel speed.

The bit-serial design also makes each PE extremely small, which is what allows 2,304 processors and their local memories to fit alongside I/O buffers, patch-level multipliers, and the ARM subsystem on a low-cost FPGA. Each patch additionally provides a conventional word-parallel ALU with DSP slices, which handles operations such as multiplication at reduced parallelism. Data movement between PEs uses nearest-neighbour shifts in the four cardinal directions (NEWS); no global interconnect or random-access bus is required. Input and output of 8-bit feature maps is handled by dedicated memory (gearbox) buffers interfaced via 32-bit AXI-Stream, providing 3.6,Gbit/s of input and 2,Gbit/s of output throughput at 165,MHz. The instruction set is summarised in Tab. 1.

This design is a work in progress and it implements on KRIA KV260 with 165MHz clock using Vivado 2025.2. Future iterations will add approximate multiplication/division, operations in log-domain and further ISA extensions.

3. Model

We implement an adapted version of STT given the constraints of the hardware described in Sec. 2. The model uses

a user-specified foveation point, provided either by the host processor or by an on-device kernel such as gaze tracking, to tokenise a (48×48) input image into 40 foveated tokens. The encoder contains approximately 201k parameters and produces a (40×64) latent token representation, which is computed in 8-bit precision on the processor array. This latent representation is then transmitted to the host, where a 231k-parameter decoder produces the final (48×48) segmentation mask.

The model is generally targeted towards low-power accelerators with limited memory and compute, and is not fixed to our specific hardware. Foveation restricts processing to task-relevant regions, while quantisation lowers arithmetic and communication cost. Although we demonstrate image segmentation, the architecture generalises to any encoder-heavy vision workload, as the processor efficiently handles convolutions, matrix multiplication and positional encoding.

3.1. Model Structure

The image is foveated around the query point onto a 16×16 patch grid, then compressed into 40 multi-resolution tokens (see Sec. 4.1). Each token is linearly embedded as a 64-dimensional vector and augmented with a learned absolute positional encoding. Six pre-norm 8-head transformer blocks produce the latent features, which are transmitted to the host for decoding (see Sec. 4.3).

3.2. Quantization

All activations, weights, and biases use symmetric 8-bit fixed-point with step size 1/64 (i.e. 6 fractional bits). STE-based fake-QAT [1, 7, 9, 11, 14] is applied to all tensors during training, covering patch embedding, positional encoding, LayerNorm, attention, and MLP layers, ensuring direct transfer to integer-only inference without calibration.

4. Implementation

For algorithm development, we wrote a Python-based instruction-set simulator using NumPy/CuPy. The simulator emits the instructions from Table 1 and models their effect on the PE array, enabling rapid kernel prototyping without RTL interaction. The same instruction stream can then be executed in a cycle-accurate RTL co-simulation via cocotb [6], using a Verilator-compiled hardware model. This allows a program developed in the Python simulator to be run against the RTL model with minimal changes, reducing the gap between algorithm exploration and hardware verification to a configuration switch.

The full model and its intermediate activations exceed the on-chip memory budget, so we stream weights from the host as each layer executes and spill less-frequently-reused intermediates back to host memory.

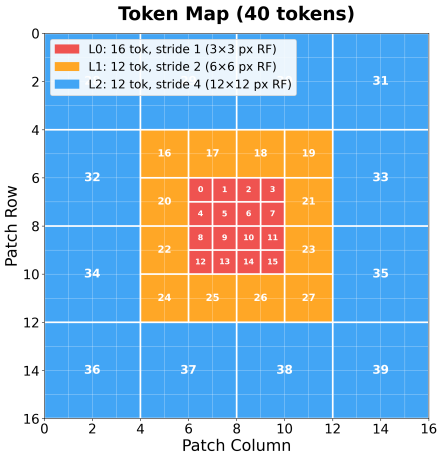


Figure 3. Foveated Tokenisation Pattern

4.1. Tokenisation

Foveated Patches We patchify the input image using a multi-resolution scheme derived from STT [12]. A stride-3 convolution maps the 48×48 input to a 16×16 feature grid, which is then partitioned into 40 tokens at three foveation levels centred on the point of interest (Fig. 3):

- **L0** (16 tokens, stride 1): a 4×4 block at full resolution surrounding the fixation point; each token covers a single 3×3 px patch.
- **L1** (12 tokens, stride 2): an intermediate ring; each token pools a 2×2 group of patches (6×6 px receptive field).
- **L2** (12 tokens, stride 4): the outer periphery; each token pools a 4×4 group of patches (12×12 px receptive field).

This achieves an 84% reduction in token count versus flat 16×16 tokenisation of the same grid, while preserving fine-grained detail near the point of interest. Peripheral tokens (L1, L2) are box-filtered before downsampling to ensure their receptive field captures the larger spatial region. SIMD predicate masks for each foveation level are precomputed and loaded based on the point prompt.

Convolutional Embedding To perform a 3×3 convolution with stride 3, we create nine shifted copies of each input channel via `NEWS` nearest-neighbour transfers (similar to [3]). The result is then given by a shift-and-add multiply-accumulate (MAC) operation against these shifted values and the quantised filter weights. The 40 foveated tokens are then obtained by averaging each token's constituent grid cells using predicated reductions.

Positional Encoding We follow STT [12] and use learned absolute positional embeddings [8], which are well suited to fixed-point arithmetic as they avoid approximating transcendental functions at limited precision. Embeddings are added to tokens before the attention blocks.

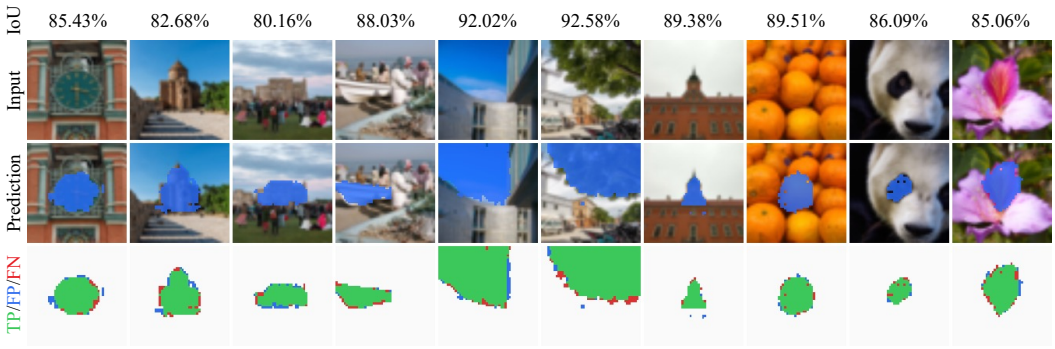


Figure 4. Examples segmented by our model. The first row shows the original input images, the second row shows predicted masks overlaid on input images, and the third row shows the true-positive (TP), false-positive (FP), false-negative (FN) area of the masks. We foveate from the centre of the images.

4.2. Attention Block

Linear Layers for Attention Tokens are multiplied by weights W_Q, W_K, W_V, and W_O. These matrices exceed the memory budget of a single patch, so we must load them in sections: each patch loads one part of the active weight matrix at a time. Partial sums are accumulated in patch memory; when on-chip storage is insufficient, immediately reused data are retained on-chip while less-frequently reused data are transmitted back to the host.

Multihead Attention Multi-head attention requires two matrix products per output head, computed as a sequence of MAC operations between rows and columns of Q, K, and V: the score matrix $S = QK^\top$ and the value aggregation $O = \mathrm{softmax}(S)V$. With one token replicated across the 36 PEs of a patch, we place a head's eight elements on eight PEs and compute the products of $q_i^\top k_j$ in a single bit-serial MAC, then sum them with a three-stage nearest-neighbour (NEWS) reduction into the patch's corner PE (≈ 32 cycles per score). Exponential and transcendental functions are not yet supported on our hardware, so we offload the softmax calculation to the host: the score matrix is streamed out, normalised in floating point on the ARM core, and the attention weights are streamed back.

4.3. Decoder on Host Processor

The latent tokens obtained from the stages described in Secs. 4.1 and 4.2 are stored in patch-local memory and then offloaded to the host processor. The host then applies a lightweight three-block SAM-style cross-attention mask decoder [10] to generate the final segmentation mask.

5. Results

5.1. Experimental Details

We instantiate the proposed architecture at a scale compatible with the KV260 FPGA: 48 × 48 input, hidden dimension 64, 8 attention heads, 6-layer encoder, 8-bit fixed-point, substantially smaller than STT [12]. The host decoder matches STT's architecture, data representation, and numerical precision. Rather than large-scale benchmarking, our goal is to verify that segmentation remains learnable under aggressive quantisation, foveation, and low resolution. We randomly sample 1,000 images from SA-1B [10], overfit to this subset using Dice and cross-entropy losses, and report mIoU on the same set. Developing a generalized performance model is in the future work.

Table 2. Comparison between STT and our model. Both are evaluated on a 1,000-image subset of SA-1B. STT is a general-purpose model evaluated zero-shot on these images, whereas our model is deliberately overfit to this same 1,000-image subset (we report training-set mIoU, without a held-out split). The comparison is included only to confirm that segmentation remains learnable under aggressive foveation, quantisation, and low resolution, and to contrast resource cost. STT uses inputs at its native 1280×1280; ours uses 48×48. FLOPs and memory are normalised per pixel to account for resolution differences; memory counts parameter storage only. The 8-bit encoder uses integer arithmetic and so is not reported in FLOPs.

	STT (FP32)	Ours (FP32)	Ours (Fixed 8)
mIoU†	$0.524^\dagger$	$0.894^\dagger$	0.714
FLOPs per pixel	9.35×10^3	1.48×10^2	–
Memory per pixel	220.0 B	51.2 B	12.8 B

† STT is zero-shot; ours is overfit to the evaluation set.

5.2. Selected Foveated Segmentations

Fig. 4 shows representative examples successfully segmented by the model. When the foveation center is placed on the target object, the model recovers the overall object contour using only 40 tokens and an 8-bit integer encoder.

5.3. Quantitative Results & Computational Cost

After 40 epochs, the model reaches the mIoU reported in Tab. 2. We stress that this figure is measured on the training set: our aim here is to verify that point-prompted segmentation remains learnable under aggressive foveation, 8-bit quantisation, and 48×48 resolution, not to establish state-of-the-art quality. The mIoU should therefore be read as a feasibility check given the hardware constraints rather than

machine learning benchmark result.

In FP32, our model requires far fewer FLOPs and less memory per pixel than STT, showing that the efficiency gains stem primarily from foveated tokenisation and reduced encoder capacity rather than from fixed-point arithmetic alone. Switching to 8-bit fixed-point reduces both further, aligning with the low-power on-sensor execution budget. Our approach occupies a different accuracy–resource trade-off point than STT: rather than maximising absolute quality, it aims to provide a simpler architecture, that can be ultimately implemented on-sensor for the presented architecture.

References

[1] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. 3

[2] Laurie Bose, Jianing Chen, Stephen J. Carey, Piotr Dudek, and Walterio Mayol-Cuevas. Visual odometry for pixel processor arrays. In *ICCV*, 2017. 1

[3] Laurie Bose, Jianing Chen, Stephen J. Carey, Piotr Dudek, and Walterio Mayol-Cuevas. A Camera That CNNs: Towards embedded neural networks on pixel processor arrays. In *ICCV*, 2019. 3

[4] Laurie Bose, Jianing Chen, and Piotr Dudek. Descriptor-In-Pixel : Point-feature tracking for pixel processor arrays. In *CVPR*, 2025. 1

[5] Stephen J. Carey, Alexey Lopich, David R.W. Barr, Bin Wang, and Piotr Dudek. A 100,000 fps vision sensor with embedded 535gops/w 256×256 SIMD processor array. In *Symp. VLSI Circuits*, 2013. 1

[6] cocotb contributors. cocotb: a coroutine-based cosimulation library for writing vhdl and verilog testbenches in python. https://github.com/cocotb/cocotb, 2024. 3

[7] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. In *ICLR*, 2020. 3

[8] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. In *ICML*, 2017. 3

[9] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *CVPR*, 2018. 3

[10] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. In *ICCV*, 2023. 4

[11] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *ICML*, 2022. 3

[12] Tanner Schmidt and Richard Newcombe. Segment This Thing: Foveated tokenization for efficient point-prompted segmentation. In *CVPR*, 2025. 1, 3, 4

[13] Haley M. So, Laurie Bose, Piotr Dudek, and Gordon Wetzstein. PixelRNN: In-pixel recurrent neural networks for end-to-end-optimized perception with neural sensors. In *CVPR*, 2024. 1

[14] Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley J. Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *ICLR*, 2019. 3

[15] Hongyi Zhang, Laurie Bose, Jianing Chen, Piotr Dudek, and Walterio Mayol-Cuevas. Focal plane visual feature generation and matching on a pixel processor array. In *ICCV*, 2025. 1